

BÁO CÁO SAU BUỔI THỰC HÀNH_SESSION13

Môn học: AI APPLICATION IN ACTION

Tên buổi thực hành: ỨNG DỤNG AI TRONG PHÂN TÍCH VÀ PHÁT TRIỂN HỆ THỐNG WEB SERVICE

Thời gian thực hành: 120p

Họ và tên sinh viên: Lê Nhật Linh

Mã sinh viên: PTIT-HN-121

Lớp: HN-K24-CNTT2

Nhóm: 2

I. NỘI DUNG ĐÃ THỰC HÀNH

1. Trình bày các giải pháp

Đề giải quyết bài toán chuyển đổi số từ quản lý bằng Excel/sổ sách sang hệ thống "Smart E-Shop" cho TechNova, em và nhóm đã áp dụng chuỗi giải pháp toàn diện phối hợp với AI:

- **Giải pháp Phân tích Nghiệp vụ (Sử dụng Role-prompting với AI Senior BA):**
 - Bóc tách tài liệu thô thành 3 Actor chính: Khách hàng (Customer), Nhân viên kho (Staff), và Quản lý (Manager).
 - Xác định rõ các Yêu cầu chức năng (FR) như: Xem sản phẩm, Quản lý giỏ hàng, Thanh toán, CRUD Sản phẩm, Xem doanh thu và Phân quyền.
 - Xác định Yêu cầu phi chức năng (NFR): Tính bảo mật thông tin, Chặn truy cập trái phép vào trang Admin, và Khả năng chịu tải hàng chục ngàn CCU (Concurrent Users)

Giải pháp Thiết kế Hệ thống (UML qua Mermaid Code):

- Sử dụng AI sinh mã Mermaid vẽ Use Case tổng quan thể hiện sự cô lập giữa phân hệ khách hàng và phân hệ Admin.
- Thiết kế ERD chuẩn hóa gồm 5 bảng cốt lõi: Users, Roles, Products, Orders, Order_Items.

Giải pháp Kỹ thuật Backend (Spring Boot 3.x & Spring Security 6):

- Bảo mật & RBAC: Sử dụng cấu hình không trạng thái (Stateless) với JWT. Phân luồng API nghiêm ngặt: /api/v1/public/ (PermitAll), /api/v1/admin/products (STAFF/MANAGER), /api/v1/admin/users (MANAGER).
- Tối ưu Hiệu năng Database: Áp dụng chiến lược nạp dữ liệu trì hoãn (FetchType.LAZY) kết hợp @EntityGraph tại tầng Repository để triệt tiêu hoàn toàn vấn đề N+1 Query khi truy vấn các mối quan hệ 1-N và N-N.
- Ràng buộc Dữ liệu: Sử dụng Jakarta Validation (@NotBlank, @Positive) tại tầng DTO để đảm bảo tính toàn vẹn dữ liệu trước khi đi vào tầng nghiệp vụ (Service).

2. Công việc nhóm:

- Tổ chức meeting nhóm cùng phân tích, thảo luận, phản biện về những yêu cầu trong mini project.
- Phân chia các tầng kiến trúc và chức năng cụ thể cho từng thành viên:
 1. Phụ trách cấu hình Spring Security, JWT và Custom Exception Handler (401/403).
 2. Phụ trách Thiết kế Database Entity, tối ưu hóa kịch bản Fetching dữ liệu phòng chống N+1, và hiện thực hóa logic nghiệp vụ tầng Service/Repository.
 3. Lập trình tầng Controller, Validate dữ liệu đầu vào DTO và tích hợp Swagger/Postman test API.

III. KẾT QUẢ CÁC EM ĐẠT ĐƯỢC

Link mini-project:

https://github.com/LINH-2006-CUTE/PTIT_CNTT2-K24_IT212_Session13/tree/main

IV. KHÓ KHĂN VÀ VẤN ĐỀ CÁC EM GẶP PHẢI

- Sự "ảo tưởng" (Hallucination) của AI trong Security: Khi prompt AI viết cấu hình Spring Security 6, AI rất hay sinh ra cú pháp cũ của Spring Security 5 (như sử dụng .authorizeRequests() thay vì .authorizeHttpRequests() hoặc .and()). Nếu không có kiến thức nền, sinh viên sẽ bị lỗi biên dịch (Compile error) ngay lập tức.
- Rủi ro Transactional khi giải quyết N+1 Query: Mặc dù áp dụng FetchType.LAZY, nếu vô tình gọi một hàm toString() hoặc log dữ liệu có chứa collection liên kết bên ngoài một @Transactional context, hệ thống sẽ ném ra lỗi LazyInitializationException. AI thường bỏ qua bối cảnh runtime này khi viết code mẫu.

V. KINH NGHIỆM RÚT RA

1. Kinh nghiệm rút ra từ bài thực hành:

Không bao giờ tin tưởng hoàn toàn vào code đầu tiên AI trả ra. Phải biết yêu cầu AI tự tìm lỗi sai trong đoạn code nó vừa viết để lọc sạch các thư viện lỗi thời hoặc lỗ hổng bảo mật

2. Lưu ý cho những buổi thực hành sau:

- Chuẩn bị tốt tất cả các phần trong yêu cầu.
- Đặt câu hỏi giúp mở rộng kiến thức và nâng cao khả năng phản biện.

VI. ĐỀ XUẤT / KIẾN NGHỊ

- Không

VII. KẾT LUẬN

1. Buổi thực hành giúp em làm chủ được quy trình từ thiết kế tài liệu nghiệp vụ đến triển khai code thực tế theo đúng chuẩn công nghiệp nhờ sự trợ lực từ AI. Việc kết hợp Spring Security 6 với JPA tối ưu giúp hình thành tư duy viết code sạch và quan tâm tới hiệu năng hệ thống ngay từ đầu.

2. Khả năng hiểu bài: Ổn